# Step 1 Summary — Reinforcement Learning Basics

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

## Contents



# 1 Step 1 — Reinforcement Learning Basics

This is a condensed summary of the foundational reinforcement learning material covered in Step 1. It is written to serve two purposes: as a quick refresher while progressing through later steps, and as a primary source for the Step 15 public report synthesis.

---

## 1.1 How Reinforcement Learning Became a Discipline

Reinforcement learning did not emerge fully formed. Its roots stretch across multiple fields that were largely unaware of each other for decades.

The earliest thread comes from **animal psychology** in the early 1900s — Thorndike's "Law of Effect" (1911) proposed that actions followed by satisfying outcomes become more likely. This is, in essence, the reward signal idea. A parallel thread ran through **optimal control theory** in the 1950s, where Bellman developed dynamic programming to solve sequential decision problems. He introduced the value function and the principle of optimality, both of which sit at the core of modern RL. A third thread came from **trial-and-error learning** in early AI research — Samuel's checkers program (1959) learned by playing against itself, adjusting its evaluation function based on wins and losses.

These threads stayed separate until the 1980s and 1990s. Sutton's PhD work on temporal-difference learning (1988) bridged the gap between animal learning theories and Bellman's dynamic programming. Then Watkins formalized Q-learning (1989), giving the field its first clean, model-free control algorithm with convergence guarantees. By the time Sutton and Barto published their textbook in 1998 (updated 2018), reinforcement learning had consolidated into a recognizable discipline with its own notation, problem taxonomy, and algorithmic toolbox.

The deep learning revolution hit RL in 2013–2015 when DeepMind's DQN learned to play Atari games from raw pixels. That demonstration made it clear that RL algorithms, combined with neural function approximators, could handle high-dimensional real-world problems. PPO followed in 2017 as a practical policy gradient method, and since then RL has expanded into robotics, game AI, recommendation systems, and language model alignment (RLHF).

The important takeaway is that RL is not just "machine learning with rewards." It is a distinct framework for sequential decision-making under uncertainty, with its own theoretical foundations drawn from control theory, psychology, and statistics.

> **Read more:** Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition — Chapter 1.
> Free: http://incompleteideas.net/book/the-book-2nd.html

---

### 1.2 Core Principles — Agent, Action, Environment

The entire RL framework rests on a simple loop. An **agent** observes the current state of an **environment**, selects an **action**, and receives a **reward** signal plus a new state. Then it repeats. The agent's goal is to learn a **policy** — a mapping from states to actions — that maximizes the cumulative reward over time.

A few terms to keep straight:

- **State** ($s$): a description of the environment at a particular moment. In CartPole, this is the cart position, cart velocity, pole angle, and pole angular velocity. In poker, it would include the cards dealt and the betting history.
- **Action** ($a$): what the agent can do. Discrete (push left or right) or continuous (apply a specific torque).
- **Reward** ($r$): a scalar feedback signal. The agent gets it after every action. Rewards can be sparse (only at the end of a game) or dense (every timestep).
- **Policy** ($\pi$): the agent's strategy. Can be deterministic ($\pi(s) = a$) or stochastic ($\pi(a|s)$ gives a probability distribution over actions).
- **Value function** ($V^\pi(s)$): the expected cumulative reward from state $s$ onward, if the agent follows policy $\pi$. This tells the agent how good a state is in the long run, not just immediately.
- **Discount factor** ($\gamma$): a number between 0 and 1 that controls how much the agent cares about future rewards versus immediate ones. $\gamma = 0.99$ means the agent is patient; $\gamma = 0.5$ means it is

myopic.

The critical design challenge in RL is the **exploration-exploitation tradeoff**. The agent must explore unfamiliar actions to discover potentially better strategies, but it also must exploit its current knowledge to collect reward. Too much exploration wastes time; too much exploitation gets stuck at local optima.

Two broad families of algorithms exist. **Value-based** methods (like DQN) learn the value of states or state-action pairs and derive a policy from those values. **Policy-based** methods (like PPO) learn the policy directly. Both have strengths; much of the RL landscape is about combining them effectively.

> **Read more:** OpenAI Spinning Up — "Key Concepts in RL"
> https://spinningup.openai.com/en/latest/spinningup/rl_intro.html

---

### 1.3 The MDP Formulation

A **Markov Decision Process** (MDP) is the formal mathematical framework that most RL algorithms are built on. It consists of five components:

- $S$: a set of states
- $A$: a set of actions
- $P(s'|s,a)$: the transition function — probability of moving to state $s'$ given state $s$ and action $a$
- $R(s,a,s')$: the reward function
- $\gamma \in [0,1)$: the discount factor

The "Markov" part means that the future depends only on the current state, not on the history of how you got there. This is a strong assumption — and one that does not hold in many interesting settings (like poker, where the history of bets matters). We deal with that in Section 4.

The objective is to find a policy $\pi^*$ that maximizes the expected discounted return:

$$\pi^* = \arg\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

The key recursive relationship is the **Bellman equation**:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma V^{\pi}(s')\right]$$

This says: the value of a state equals the expected immediate reward plus the discounted value of wherever you end up. All of dynamic programming, TD learning, and deep RL build on variations of this equation.

> **Read more:** Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 3 — Finite Markov Decision Processes.
> Free: http://incompleteideas.net/book/the-book-2nd.html

---

### 1.4 Information Sets — When You Cannot See Everything

Standard MDPs assume the agent can fully observe the state. Real problems often violate this. In poker, you do not see the opponent's cards. In real-time strategy games, you have fog of war. These are **partially observable** settings.

The formal extension is the **Partially Observable MDP** (POMDP), which adds an observation function — the agent sees observations that are noisy or incomplete views of the true state. But POMDPs are notoriously hard to solve optimally.

In game theory, the same idea is captured through **information sets**. An information set groups together all game states that a player cannot distinguish between, given what they have observed. In Kuhn Poker (3 cards, 2 players), when you hold a Jack and no betting has occurred, you are in an information set — you know your card but not your opponent's, so you cannot tell if the true state is "I have Jack, opponent has Queen" or "I have Jack, opponent has King."

Algorithms that work in these settings — like CFR (Counterfactual Regret Minimization, covered in Step 2) — operate over information sets rather than individual states. This is a fundamentally different computational problem than standard RL, and it is why game-playing AI requires its own set of tools beyond what DQN or PPO provide out of the box.

Understanding information sets now matters because the later steps (5–8) deal entirely with imperfect-information games. The algorithms change, but the core concept stays: you are making decisions based on what you can observe, reasoning about what you cannot.

> **Read more:** Shoham, Y. & Leyton-Brown, K. (2008). *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*, Chapter 5.
> Free: http://www.masfoundations.org/download.html

---

### 1.5 Dynamic Programming — When You Have the Model

Dynamic programming (DP) is the oldest approach to solving MDPs. It works when you have complete knowledge of the environment: the transition probabilities $P(s'|s, a)$ and the reward function $R(s, a, s')$. In practice, this means you know the rules of the game perfectly.

The two main DP algorithms are:

**Policy Iteration** alternates between two steps. First, *policy evaluation*: given a fixed policy, compute the value of every state by repeatedly applying the Bellman equation until convergence. Second, *policy improvement*: update the policy to be greedy with respect to the newly computed values. Repeat until the policy stops changing. It is guaranteed to converge to the optimal policy.

**Value Iteration** is a shortcut. Instead of fully evaluating a policy before improving it, it combines evaluation and improvement into a single update. At each step, for every state, pick the action that maximizes the expected one-step return plus the discounted value of the next state. It converges to the optimal value function, from which the optimal policy is extracted.

DP is beneficial because it gives exact solutions and has clean convergence guarantees. The downside is obvious: you need the full model, and the computation scales with the number of states. For Kuhn Poker with 12 states, DP is trivial. For Go with $10^{170}$ states, it is impossible.

The reason DP matters even though we rarely use it directly is that every subsequent method — Monte Carlo, TD learning, DQN, PPO — can be understood as an approximation to the DP solution when the model is unknown or the state space is too large. DP is the theoretical ceiling that practical algorithms try to approach.

> **Read more:** Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 4 — Dynamic Programming.
> Free: http://incompleteideas.net/book/the-book-2nd.html

---

### 1.6 Temporal-Difference Learning — Learning Without a Model

Temporal-difference (TD) learning is the breakthrough that made RL practical. Unlike DP, it does not require a model of the environment. Unlike Monte Carlo methods, it does not have to wait until the end of an episode to learn. It updates value estimates after every single step, using a bootstrap: the current estimate of the next state's value stands in for the true future return.

The basic TD update (TD(0)) for the value function is:

$$V(s_t) \leftarrow V(s_t) + \alpha \left[ r_t + \gamma V(s_{t+1}) - V(s_t) \right]$$

The term in brackets is the **TD error** — the difference between what happened (reward + estimated future value) and what was predicted. If the TD error is positive, the state was better than expected; if negative, worse.

The control version of this idea is **Q-learning** (Watkins, 1989):

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[ r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

Q-learning learns the value of state-action pairs and uses the max operator to be **off-policy** — it learns about the optimal policy even while following an exploratory one. This is what makes it so practical.

TD learning is beneficial compared to DP because it needs no model. It is beneficial compared to Monte Carlo because it learns online (every step, not every episode) and has lower variance. The bootstrapping

introduces some bias, but in practice this tradeoff is overwhelmingly favorable. TD methods are the foundation that DQN and most modern value-based algorithms are built on.

**SARSA** is the on-policy cousin of Q-learning — it updates using the action the agent actually took next, rather than the best possible action. This makes it more conservative and sometimes safer in practice, but it cannot learn an optimal policy while following a suboptimal exploratory policy.

> **Read more:** Sutton & Barto (2018). *Reinforcement Learning: An Introduction*, Chapter 6 — Temporal-Difference Learning.
> Free: http://incompleteideas.net/book/the-book-2nd.html

---

### 1.7 DQN — Deep Q-Networks

DQN (Mnih et al., 2015) is the algorithm that proved deep neural networks could work as function approximators in RL at scale. Before DQN, Q-learning was limited to tabular settings or hand-crafted features. DQN showed it could learn to play 49 Atari games from raw pixel input, surpassing human performance on many of them.

**How it works.** DQN replaces the Q-table with a neural network that takes a state as input and outputs Q-values for all actions. The network is trained by minimizing the squared TD error: the difference between the predicted Q-value and the target (reward + discounted max Q-value of the next state). At each step, the agent picks the action with the highest Q-value (exploitation) or a random action with probability $\epsilon$ (exploration).

Two tricks make this stable enough to work:

1. **Experience replay.** Instead of training on sequential transitions (which are highly correlated), transitions are stored in a large buffer and sampled randomly in mini-batches. This breaks the correlation, reduces variance, and lets each experience be reused multiple times. It is the same principle as shuffling your training data in supervised learning.

2. **Target network.** A separate copy of the Q-network is maintained and updated only periodically (every few thousand steps or episodes). The TD target is computed using this frozen copy. Without it, the target shifts with every training step — you are chasing a moving target, which causes oscillation and divergence. The target network decouples the update and stabilizes learning.

**Why DQN matters compared to what came before:** tabular Q-learning cannot handle high-dimensional states (images, continuous observations); function approximation with Q-learning alone was known to be unstable (the "deadly triad" of off-policy + function approximation + bootstrapping). DQN's two tricks — replay and target networks — addressed the instability, making deep value-based RL practical for the first time.

**Limitations.** DQN handles only discrete action spaces. It can overestimate Q-values (addressed by Double DQN). It is off-policy, which is sample-efficient but can lead to stale data in the replay buffer.

And fundamentally, it learns a greedy deterministic policy — there is no natural way to express stochastic policies, which matters in games where mixing strategies is optimal.

In our Step 1 implementation, DQN solved CartPole-v1 (achieving a 100-episode average of 477.5 against a 475 target) after ~1000 episodes of training. The most impactful hyperparameters were network size, epsilon minimum, and target network sync frequency.

**A note on neural networks in this step.** The Q-network inside DQN is a standard multi-layer perceptron (MLP) used here as a practical tool — a function that maps states to Q-values. For Step 1, the neural network internals (weight initialization, gradient flow, activation functions) were treated as a black box: we used PyTorch's defaults and focused on the RL algorithm itself. A more thorough treatment of neural network architectures and how they interact with equilibrium-finding methods will come in Step 5 (Neural Equilibrium Approximation).

> **Read more:** Mnih, V. et al. (2015). "Human-level control through deep reinforcement learning." *Nature*, 518(7540), 529–533.
> arXiv: https://arxiv.org/abs/1509.06461

---

### 1.8 PPO — Proximal Policy Optimization

PPO (Schulman et al., 2017) takes a fundamentally different approach to RL than DQN. Instead of learning the value of actions and deriving a policy from those values, PPO **learns the policy directly** — a neural network outputs a probability distribution over actions, and the network is trained to increase the probability of actions that lead to high returns.

**The problem PPO solves.** Policy gradient methods are powerful in theory but notoriously unstable in practice. The basic REINFORCE algorithm suffers from high variance and can take destructively large steps that ruin a good policy. Trust Region Policy Optimization (TRPO, Schulman et al., 2015) addressed this by constraining how much the policy can change per update, but TRPO requires expensive second-order optimization (computing the Hessian).

**How PPO works.** PPO keeps the stability idea from TRPO but replaces the constraint with a much simpler mechanism: **clipping**. The key quantity is the probability ratio $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$ — how much more (or less) likely the new policy is to take the same action the old policy took. PPO clips this ratio to the range $[1-\epsilon, 1+\epsilon]$ (typically $\epsilon = 0.2$), which means:

- If an action turns out to be good (positive advantage) and the ratio grows beyond $1+\epsilon$, the gradient is zeroed — the policy is not allowed to become too eager for that action.
- If an action turns out to be bad (negative advantage) and the ratio drops below $1-\epsilon$, the gradient is also zeroed — preventing the policy from overcorrecting.

This is the **clipped surrogate objective**, and it is what makes PPO work. It gets TRPO-like stability using only first-order gradients (no Hessian), making it substantially cheaper to compute and easier to

implement.

**Architecture.** PPO is actor-critic: a **policy network** (actor) outputs action probabilities, and a **value network** (critic) estimates state values. The value network enables Generalized Advantage Estimation (GAE), which provides low-variance advantage estimates for the policy gradient. Both networks are trained simultaneously.

**Why PPO matters compared to DQN:** PPO naturally handles both discrete and continuous action spaces. It can learn stochastic policies, which is critical in game settings where mixing strategies is necessary. It is on-policy (uses fresh data from the current policy), which avoids stale data issues. And the clipped objective makes it remarkably robust — PPO is the default choice for most RL applications today, including RLHF for language models.

**Limitations.** Being on-policy, PPO is less sample-efficient than off-policy methods like DQN (each experience is used for only a few update epochs before being discarded). It requires careful tuning of the advantage estimation (GAE lambda), entropy bonus, and network architecture. And it can still struggle with sparse rewards — if the agent rarely encounters success, there is little signal to learn from.

In our Step 1 implementation, PPO solved LunarLander-v3 (100-episode average of 202.2 against a 200 target) at ~264K environment steps. The decisive factors were network capacity (doubling from [64,64] to [128,128]) and advantage normalization.

**A note on neural networks in this step.** Same as with DQN — the policy and value networks here are standard MLPs whose internals were not the focus of this step. We adopted a practical "it works" stance toward the neural components and concentrated on understanding the PPO algorithm logic (clipping, GAE, rollout structure). The deeper interplay between neural network design and RL convergence — particularly in the context of game-playing agents — is deferred to Step 5.

> **Read more:** Schulman, J. et al. (2017). "Proximal Policy Optimization Algorithms." arXiv: https://arxiv.org/abs/1707.06347

---

## 1.9 Practical Validation — Custom Implementations vs Stable-Baselines3

To verify that our from-scratch implementations actually work, we compared them against Stable-Baselines3 (SB3) — a widely used, well-tested RL library. Both our implementations and SB3 were given matched hyperparameters and equivalent training budgets.

### 1.9.1 DQN on CartPole-v1

Our custom DQN solves CartPole by episode ~1011, reaching a 100-episode rolling average of 477.5 (above the 475 target). SB3's DQN, given 750K environment steps (17,176 episodes), never converges — its rolling average stays around 30 and never approaches the target.

The gap comes from three implementation-level differences:

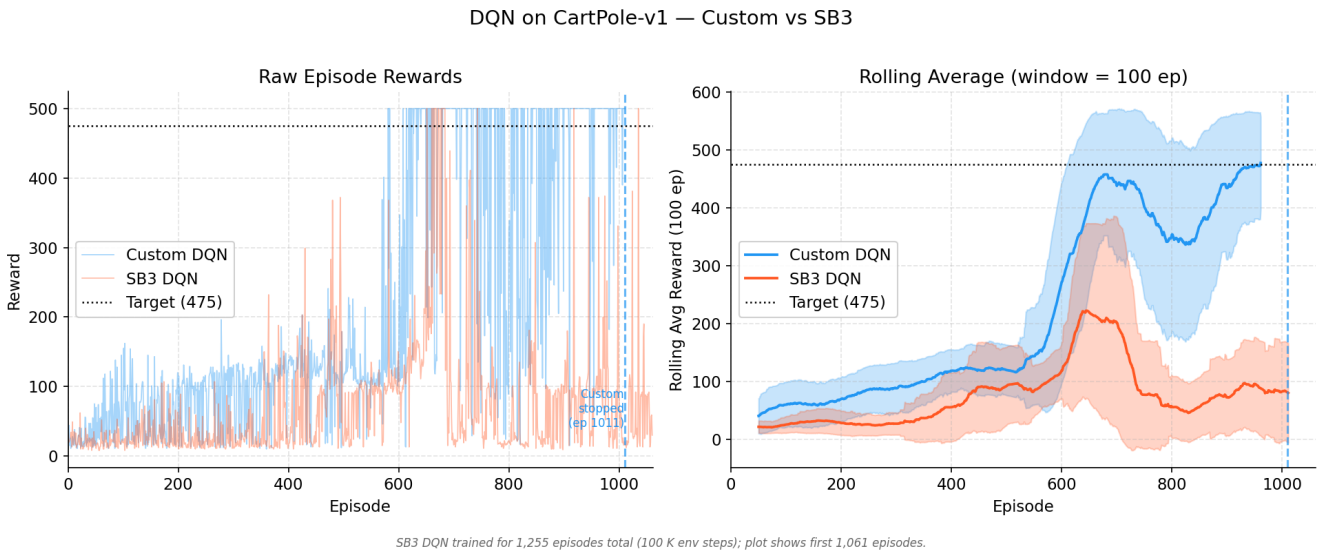


Figure 1: DQN comparison — Custom vs SB3

- **Exploration schedule.** Our epsilon decays per episode (×0.995 each), concentrating exploration where it matters. SB3 decays epsilon linearly over steps, which in short-episode environments like CartPole wastes many episodes in near-random mode.
- **Target network sync.** Our code syncs every 5 episodes — an adaptive schedule that gives faster feedback during early learning. SB3 uses a fixed 1000-step interval.
- **Early stopping.** Our agent saves the best model and stops at peak performance, avoiding catastrophic forgetting. SB3 runs the full budget, and the policy oscillates.

These are genuine algorithmic choices, not hyperparameter unfairness. SB3's defaults are designed for robust performance across diverse, long-training domains like Atari games — not brief episode classic control environments like CartPole; our task-specific choices simply work better on CartPole.

### 1.9.2 PPO on LunarLander-v3

Our custom PPO crosses the 200 target around episode 543 (264K steps). SB3's PPO with the same 500K-step budget peaks at a best rolling-100 average of 131.2 — still climbing but not converged.

The gap is narrower here than with DQN. Both use identical PPO hyperparameters. The remaining difference likely comes from SB3's orthogonal weight initialization and built-in observation normalization, which help long runs but may slow early convergence. Given more training budget (1–2M steps), SB3 PPO would likely reach the target.

### 1.9.3 Takeaway

A clean from-scratch implementation is 300–500 lines versus SB3's ~10K. The tradeoff: fewer features but complete transparency and debuggability. For research purposes — particularly the opponent exploitation work planned for Steps 7–8 — custom implementations allow surgical modifications (e.g., injecting belief-

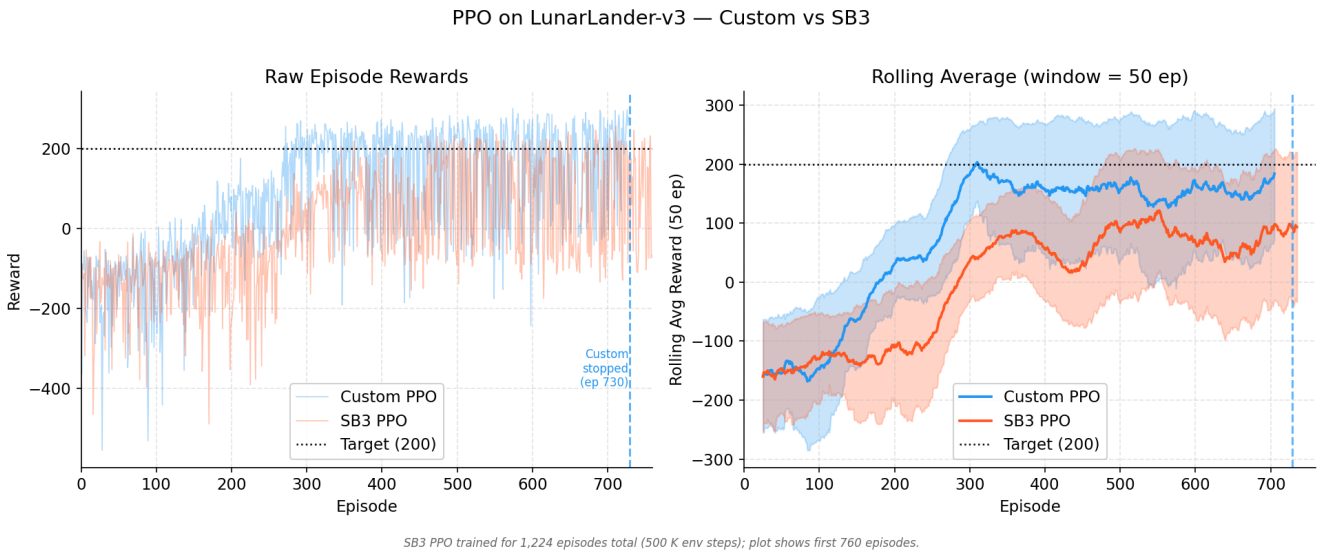


Figure 2: PPO comparison — Custom vs SB3

state observations) that would require deep SB3 subclassing. That practical flexibility is why we built from scratch.